课程编号：

《开源创新思维》
平时作业报告

姓　　名	徐悦	学　　号	f25016515
班　　级	f250165	指　导　教　师	祝欣蓉
开　设　学　期	2026-2027 秋 季 学 期		
开　设　时　间	第 1 周 — — 第 16 周		
报　告　日　期	2026 年 10 月 29 日		
评　定　成　绩		评　定　人	祝欣蓉
		评　定　日　期	

上海杉达学院

1、扫描对象：Apache Commons Lang 3.21.0

本次选取 Apache 软件基金会旗下的 Apache Commons Lang 项目，版本 3.21.0。扫描对象为官方发布的源码发行包 commons-lang3-3.21.0-src.tar.gz（1505249 字节），从 Apache 官方归档站点 archive.apache.org 下载，并用同目录的 .sha512 校验文件核对完整性，计算结果与官方值完全一致（38b6615e…275e2），确认文件未被篡改，可作为合规扫描的可信输入。

Apache Commons Lang 是 Apache Commons 系列中使用最广的子项目之一，目标是在 JDK 标准库之外提供常用工具类，包括 StringUtils（字符串处理）、ArrayUtils（数组操作）、DateUtils 与 DateFormatUtils（时间处理）、ObjectUtils、NumberUtils、ReflectionToStringBuilder 等。它长期被 Spring、Hibernate、Log4j 等主流框架间接依赖，属于典型的被广泛分发的基础库，其许可证状况会沿依赖链向下游传递，因此具有代表性。

项目许可证为 Apache License 2.0。源码包根目录下同时存在 LICENSE.txt（许可证全文）与 NOTICE.txt（归属声明），符合 AL2.0 第 4 条对再分发的要求。解压后共 706 个文件，其中 Java 源文件 629 个，其余为 pom.xml、站点文档、图片与测试数据；目录以 src/main/java（主代码）、src/test/java（测试代码）、src/site（项目站点与发布说明）为主。

选择该项目的理由有三点：一是 Apache 顶级项目，头部声明规范统一，便于验证检测工具本身的判断准确性；二是许可证单一（仅 AL2.0），不存在多许可证混合的干扰因素，结果可解释；三是体量适中（706 个文件），单次扫描十余秒即可完成，适合反复验证。

表 1　扫描对象基本信息

项目	内容
项目名称	Apache Commons Lang
版本	3.21.0
发行包	commons-lang3-3.21.0-src.tar.gz（1505249 字节）
完整性校验	SHA-512 与官方 .sha512 一致
许可证	Apache License 2.0
文件规模	706 个文件，其中 Java 源文件 629 个

2、检测工具的安装与遇到的问题

选用 Apache Creadur RAT（Release Audit Tool）0.18 作为检测工具。它由 Apache 基金会自己维护，专用于审计源码树中的许可证头部与版权声明，与本作业的场景高度契合。

安装过程：RAT 官方文档默认给出的用法是 Maven 插件（mvn apache-rat:check），但本机未安装 Maven（执行 mvn -v 返回 command not found），因此改用免安装的独立命令行包。从 Maven 中央仓库下载 apache-rat-0.18.jar（7479290 字节），该 jar 已把依赖一并打包，MANIFEST.MF 中 Main-Class 为 org.apache.rat.Report，可直接以 java -jar 方式运行，无需解压或配置环境变量。运行环境为 Windows + JDK 17.0.18。

安装与首次运行过程中共遇到 5 个问题，逐一记录在表 2 中。其中第 2 条最值得注意：RAT 把“检出问题”与“运行失败”绑定在一起，默认阈值下只要有一个文件不通过，进程就以异常结束、不输出报告文件（第一次运行后 out 目录为空）。这与其他扫描工具“先出报告、再由使用方判定”的行为不同，初次使用很容易误以为工具不可用。

表 2　安装与运行中遇到的问题及解决方式

序号	现象	原因与解决
1	mvn -v 提示 command not found	本机无 Maven，官方 Maven 插件用法不可用；改用独立 jar，以 java -jar 直接运行
2	首次扫描异常退出，且未生成报告文件	ERROR: Unexpected count for UNAPPROVED, limit is [0,0]，随后抛 RatDocumentAnalysisException；0.17 起默认要求 UNAPPROVED 计数为 0，检出即判失败。用 --counter-max 放宽上限
3	--counter-max 报 NumberFormatException	写成 --counter-max "UNAPPROVED:-1,UNKNOWN:-1" 时解析出 -1,UNKNOWN 报错；该参数不支持逗号并列，须重复传参
4	--output-file 报 FileNotFoundException	传 Windows 绝对路径时被当作相对路径拼接成 ...\tools\C:\Users\...（文件名、目录名或卷标语法不正确）；改用相对路径 out/rat-report.txt
5	报告部分中文显示为 ?????	License Categories 一节出现问号；输出编码固定，按 GBK 读取即乱码，按 UTF-8 打开可正常显示

第 3 条的实测报错为 NumberFormatException: For input string: "-1,UNKNOWN"；第 4 条改用相对路径后立即成功，可见两处均为 RAT 0.18 命令行在 Windows 下的参数处理缺陷，而非环境配置问题。

3、工具输出的检测报告

修正上述问题后，最终执行的扫描命令如下（工作目录为 hw1）：

```
java -jar tools/apache-rat-0.18.jar --counter-max UNAPPROVED:-1 \
    --counter-max UNKNOWN:-1 --output-file out/rat-report.txt \
    -- src/commons-lang3-3.21.0-src
```

扫描 706 个文件，耗时约 13.7 秒（JDK 17.0.18，单线程）。报告 Summary 部分的计数器统计见表 3。

表 3　RAT 输出报告的计数器统计

计数器	数值	含义
Approved	692	通过许可证审查的文件数
Standards	694	被识别为标准文本的文件数
Binaries	9	图片、.class、.xcf、.cgi 等二进制文件
Notices	3	LICENSE.txt、NOTICE.txt、RELEASE-NOTES.txt

计数器	数值	含义
Unapproved	2	许可证声明未通过审查的文件
Unknown	2	无法判定类型的文件
Ignored / Archives	0 / 0	未被排除规则忽略；未解包任何归档文件

检出许可证分布为：Apache License 2.0 共 692 个，Unknown license 共 2 个。报告给出的未通过清单只有两条：

```
/src/site/resources/release-notes/RELEASE-NOTES-3.9.txt
/src/test/resources/lang-708-input.txt
```

报告 Detail 部分逐文件标注了类型与检出的许可证，通过项不带前缀，未通过项以感叹号开头，示例如下：

```
 /pom.xml
 S        application/xml    ISO-8859-1
   AL      AL2.0        Apache License 2.0
! /src/test/resources/lang-708-input.txt
 S        text/plain         ISO-8859-1
   ????    ????         Unknown license (Unapproved)
```

4、检测报告的详细分析

4.1 覆盖率与合规率

RAT 对 706 个文件全部完成了分类，Ignored 计数为 0，说明没有任何文件因排除规则被跳过，扫描覆盖率 100%。其中 694 个标准文件属于需要许可证声明的对象，692 个通过，合规率 99.71%。从工程视角看，一个成熟 Apache 项目的头部规范度接近满分，剩余 0.29% 的偏差集中在非代码文件上。

4.2 两个未通过文件的定性分析

第一个是 RELEASE-NOTES-3.9.txt（63385 字节），位于 src/site/resources/release-notes/，内容是项目站点上 3.9 版本的历史发布说明，属于纯文档。第二个是 lang-708-input.txt（6038 字节），位于 src/test/resources/，内容是地理名称多语言词条的 JSON 测试数据。

二者的共同点是“数据或文档，而非可执行代码”。AL2.0 对再分发的核心义务是随分发物附带 LICENSE 与 NOTICE，并保留版权与归属声明；这两个文件不引入第三方代码，也不改变整体许可证，因此不构成实质性许可风险，风险等级判定为低。但它们确实违反了“每个文件都应有许可证头部”的内部规范，属于规范性瑕疵而非合规性缺陷。

若项目要求流水线全绿（Unapproved 为 0），可用排除规则处理。实测加入 --input-exclude "**/release-notes/**" 与 --input-exclude "**/lang-708-input.txt" 后重新扫描，扫描文件由 706 降到 668（排除 38 个文件），Unapproved 归零，进程退出码为 0，可直接作为 CI 门禁使用。

4.3 对照实验：工具是否“只认 Apache”

为了验证工具的真实判断能力，而不只是看它在一棵本身就很规范的代码树上“全绿”，另外构造了 5 个对照样本单独扫描：Apache-2.0 头部、MIT 头部、GPL-3.0 头部、无头部 Java 文件、无头部 CSV 数据文件各一个，结果如下：

（1）Apache-2.0 头部识别为 AL2.0，通过；（2）MIT 头部识别为 MIT，通过；（3）GPL-3.0 头部识别为 GPL3，但标记 Unapproved（拒绝）；（4）无头部 Java 文件与无头部 CSV 均判为 Unknown license，标记 Unapproved。

可见 RAT 的判定是“识别 + 白名单比对”两段式：先认出属于哪种许可证，再按组织策略决定是否放行。GPL3 被正确识别却拒绝，恰好说明它做的是策略判断，而不是单纯的“缺头部检测”。再用 --licenses-approved GPL3 显式把 GPL3 加入白名单，同一批样本的 Approved 由 2 变为 3、GPL3 转为通过，证明白名单可按需配置。

另一个可复现的细节：MIT 头部第一次只写了许可正文、漏掉末尾的免责声明段，RAT 判为 Unknown license；补全免责声明后才识别为 MIT。这说明它依赖完整的声明文本匹配，片段式引用（例如只写“MIT License + Copyright”）不会被认出，实践中不能靠简化头部蒙混过关。

4.4 工具的局限与盲区

（1）只做头部文本匹配，不判断许可证兼容性。AL2.0 与 GPL-3.0 并存时的传染性冲突，RAT 不会给出任何提示，这类判断必须由人工或法务完成。

（2）不解析依赖树。pom.xml 中声明的第三方依赖及其传递依赖不在扫描范围内，而实际项目中大部分许可风险恰恰来自依赖侧。本次 692 个通过文件只覆盖项目自身代码，依赖侧需配合 OSS Review Toolkit、ScanCode、FOSSLight 等 SCA/SBOM 工具补齐。

（3）不解包归档。本次 Archives 计数为 0，若项目把 jar 或压缩包提交进代码仓库，其内部文件不会被检查，存在盲区。

（4）对无头部的数据与文档天然“误报”。这类文件会长期落在 Unknown 里，必须依靠排除清单维护，否则告警会一直存在并逐渐被忽视，形成告警疲劳。

（5）命令行在 Windows 下有路径与参数语法两处缺陷（见第 2 节第 3、4 条），需要在脚本里按本文记录的方式规避，否则自动化会静默失败。

4.5 改进建议与结论

针对本项目落地，建议三点：一是把排除清单固化成 .rat-excludes 文件，将 release-notes、测试数据、图片与 .class 纳入，避免每次手工传参；二是把 apache-rat:check 接入 CI 作为发布前门禁，同时用 SBOM 工具补齐依赖侧扫描；三是对 GPL/AGPL 类依赖单独建立兼容性审查流程，不能只依赖 RAT 的白名单结果。

结论：Apache Commons Lang 3.21.0 的源码发行包在其 Apache-2.0 框架下是自洽的，706 个文件全部可解释，2 处未通过均为文档与测试数据，无实质性许可风险。RAT 作为零依赖、轻量的头部审计工具，适合作为合规检查的第一道筛查，但不能替代依赖分析与许可证兼容性判断。

评价表格：

考核标准	得分
（1）报告格式规范、语言通顺（10%）；	
（2）质量标准分析合理（10%）；	
（3）测试方法分析全面、合理（20%）；	
（4）测试风险分析全面、合理（20%）；	
（5）测试总体测试策略全面、合理（30%）	
（6）测试充分性判断标准合理（10%）	